

Manual para Iniciar un Nuevo Proyecto basado en la Arquitectura de 3 Capas

Agente IA (Orquestador)

29 de junio de 2026

Índice

1. Introducción	2
2. Estructura de Directorios	2
3. Paso 1: Archivos del Agente (.agent/)	2
4. Paso 2: Configuración del Agente (opencode.json)	3
5. Paso 3: Crear las Primeras Directivas (directives/)	3
6. Paso 4: Crear los Scripts de Ejecución (execution/)	3
7. Paso 5: Archivos de Configuración	4
7.1. .gitignore	4
7.2. .env	4
8. Flujo de Trabajo del Agente	4
9. Buenas Prácticas	5
10. Conclusión	5

1. Introducción

Este manual describe los pasos para iniciar un nuevo espacio de trabajo bajo la arquitectura de 3 capas (Directives → Orchestration → Execution). El objetivo es proporcionar una estructura reutilizable que maximice la fiabilidad, separando la lógica probabilística del LLM de la ejecución determinista mediante scripts especializados.

2. Estructura de Directorios

Crear la siguiente estructura en la raíz del nuevo proyecto:

```
proyecto/  
|-- .agent/           # Instrucciones del sistema para el agente IA  
|-- directives/       # SOPs en YAML (Layer 1)  
|-- execution/        # Scripts deterministas (Layer 3)  
|-- .tmp/             # Archivos temporales (regenerables)  
|-- .env              # Credenciales y configuracion (no committear  
    )  
|-- opencode.json     # Configuracion del agente  
|-- .gitignore        # Exclusiones de seguimiento  
|-- requirements.txt   # Dependencias (se completa al definir el  
    proyecto)
```

3. Paso 1: Archivos del Agente (.agent/)

Copiar los archivos `FRAMEWORK.md` e `INSTRUCTIONS.md` desde la plantilla ubicada en `docs/AGENTE_IA/` a la carpeta `.agent/` del nuevo proyecto. Estos archivos definen el marco operativo y las instrucciones del agente.

Personalizar los siguientes puntos en `INSTRUCTIONS.md`:

- **Sección 1:** Ajustar el tono, las restricciones de entorno y el stack tecnológico (SO, gestor de paquetes, lenguajes).
- **Sección 8:** Definir el formato de documentación del proyecto (Markdown, LaTeX, etc.).
- **Sección 10:** Completar las directrices de dominio técnico: lenguaje principal, frameworks, infraestructura, seguridad y flujo de trabajo.

En `FRAMEWORK.md`:

- **Available Directives:** Listar los flujos de trabajo repetibles propios del proyecto.
- **Setup:** Documentar los pasos de instalación específicos del proyecto.

4. Paso 2: Configuración del Agente (opencode.json)

Crear `opencode.json` en la raíz del proyecto para que el agente cargue las instrucciones desde `.agent/`:

```
{
  "$schema": "https://opencode.ai/config.json",
  "instructions": [".agent/*.md"]
}
```

5. Paso 3: Crear las Primeras Directivas (directives/)

Cada directiva es un archivo YAML que describe un flujo de trabajo repetible. Estructura recomendada:

```
# directives/mi_flujo.yaml
goal: "Descripcion del objetivo"
required_inputs:
  - name: "entrada_1"
    description: "Que datos se necesitan"
steps:
  - script: "execution/mi_script.py"
    mapping:
      entrada_1: "{entrada_1}"
expected_outputs: "Descripcion del resultado esperado"
edge_cases:
  - "Que hacer si algo falla"
```

Crear al menos una directiva inicial para validar que el flujo funciona. Ejemplos comunes: `scrape_website.yaml`, `research_topic.yaml`, `git_update.yaml`.

6. Paso 4: Crear los Scripts de Ejecución (execution/)

Cada script debe cumplir:

- Una sola responsabilidad.
- Entradas por argumentos CLI.
- Secretos en `.env`.
- Salida por stdout (preferiblemente JSON).
- Códigos de salida: 0 (éxito), 1+ (fallos categorizados).
- Validar sus propias salidas y fallar ruidosamente si algo está mal.

Ejemplo mínimo en Python:

```
#!/usr/bin/env python3
"""Descripcion breve de la funcion del script."""
import sys

def main():
    # Validar entradas
    # Ejecutar logica
    # Validar salida
    print('{"status": "ok"}')
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

7. Paso 5: Archivos de Configuración

7.1. .gitignore

Crear un .gitignore genérico:

```
--pycache--/
*.pyc
.env
.tmp/
chroma_db/
*.aux
*.log
*.out
*.toc
venv/
.venv/
```

7.2. .env

Plantilla para credenciales:

```
# API Keys
# API_KEY=tu_clave_aqui
```

8. Flujo de Trabajo del Agente

Una vez configurada la estructura, el agente opera siguiendo este ciclo:

1. **Validación de Entorno:** Verificar que el sistema cumple los requisitos.
2. **Búsqueda:** Revisar `directives/` para ver si existe un SOP para la tarea solicitada.
3. **Planificación:** Seleccionar la directiva y el script de ejecución correspondiente.

4. **Ejecución:** Invocar el script con las entradas validadas.
5. **Validación:** Confirmar que la salida coincide con lo esperado.
6. **Registro:** Guardar el estado en `.tmp/run_state.json`.
7. **Notificación:** Informar al usuario del resultado.
8. **Aprendizaje:** Si surge un error, documentarlo y actualizar la directiva.

9. Buenas Prácticas

- **Reutilizar antes de construir:** Revisar `execution/` antes de crear un nuevo script.
- **Directivas atómicas:** Cada directiva debe describir un solo flujo de trabajo.
- **Versionado de directivas:** Si cambia la lógica, conservar versiones antiguas (v1, v2).
- **Máximo 3 reintentos:** Si un flujo falla 3 veces, escalar al usuario.
- **Validar siempre:** Después de cada paso, confirmar esquema, cantidades y archivos generados.
- **Documentar errores:** Mantener un registro de errores comunes y sus soluciones (ej. `errores.md`) para evitar regresiones.

10. Conclusión

Con esta estructura, el nuevo proyecto queda listo para operar bajo la arquitectura de 3 capas. El esqueleto inicial es mínimo pero escalable: las directivas, scripts y configuraciones se agregan según el proyecto va tomando forma.